

Inayat

Judge & Faculty Q&A — Technical Deep-Dive · SIH26047, Patient Case-Taking Software

For the team, not for the panel. The companion lay-audience guide (*Inayat_Judge_Interview_Guide.pdf*) covers the simple version of these same points. This one is written for BOTH rooms this prototype will actually be shown in: a campus review in front of faculty/HOD, who open with basics - the problem, the objective, the tech stack, "is this really your own work," "how did you test it," "what's left to do" - before anything technical; and a technical SIH-style panel that goes straight for design parameters and adversarial detail. Section 0 below is the campus-viva layer; Sections 1-4 go deeper for whoever keeps asking. Every answer was checked against the real file it cites while this was written, not recalled from memory - if a claim here can't be pointed at a real line of code, it isn't in here.

Read this once, then close it. If you're reading from it in front of a judge, that's already the wrong signal - every answer here should be something you can say in your own words, because you actually built it.

0. Campus basics — what faculty/HOD ask first, in a viva, before any code

Plain-language answers on purpose. No jargon a non-CS panel member would have to stop and ask about.

Q: What problem does this solve, in one line?

A: In most government/rural hospitals a doctor gets 2-5 minutes per patient - not enough time to take a proper history or read old prescriptions. Inayat does that history-taking before the patient enters the room, so the doctor's few minutes go to deciding, not to asking the same questions again.

Q: Which problem statement is this for, and is it a real one?

A: SIH26047, "Patient Case-Taking Software," under the Ministry of Ayush - a real, currently listed problem statement, not a topic we invented. We have the official PS text cross-checked against a real screenshot of the sih.gov.in portal itself, not a third-party summary.

docs/sih/SIH26047_Patient_Case_Taking_Software.md

Q: What is the objective and scope of the project - what does it promise to do, and what does it not try to do?

A: Objective: take a patient's history (typed, spoken, or from an old prescription/report photo) and turn it into a clean, structured summary a doctor can read in seconds, plus an independent safety check that a serious symptom is never missed. Scope boundary, stated on purpose: it never diagnoses and never prescribes - the doctor makes every decision. Full hospital-record integration (ABDM) and a scored Ayurvedic assessment are explicitly first steps, not finished modules - said plainly in section 3 below.

Q: What technologies did you use, and why these and not something else?

A: Python/FastAPI for the backend (a small, fast, well-documented framework - no reason to reach for anything heavier for one focused API); plain HTML/JavaScript for the frontend, deliberately with no framework, so it runs on a low-end phone browser with no install and no build step, exactly the rural-kiosk constraint this problem statement describes; SQLite for storage, since one well-defined table doesn't need a full database server; real government tools where they already exist for this exact job (Bhashini for Indian-language voice, ABDM for the national health ID) instead of reinventing them; Anthropic's Claude for the language-understanding step, with a working fallback when no AI key is available (see below).

Q: Is this your own original work, or built on top of someone else's project/tutorial?

A: Original, built for this problem statement. It does use real, standard open-source building blocks the way any serious software project does - a web framework (FastAPI), a scikit-learn text-similarity function, Tesseract for OCR, PocketSphinx/espeak-ng for offline voice - the same way a civil-engineering project uses standard structural formulas rather than deriving them from scratch. The system design, the clinical question logic, the safety-net architecture, and every integration between these pieces is ours, and it's ours to explain line by line, which is the actual test of originality in a viva.

Q: What exactly is the "AI" doing here, and what is the doctor still responsible for?

A: The AI drafts: it turns a patient's words into a structured note, and suggests a possible urgency level. The doctor decides: every AI-drafted summary starts marked "not yet reviewed," and there's a real login-protected screen where a doctor reads it, edits anything wrong, and signs off - only then is it final. That's not a policy on a slide; it's a field in the database that starts False and only a doctor's own action can flip.

app/schemas.py: ClinicalHistorySummary.is_reviewed_by_physician

Q: How do you know it actually works - what's your testing approach?

A: Two levels. First, automated: 358 automated tests currently pass, covering the individual pieces (does the safety-word-scanner catch "chest pain," does the document reader correctly flag an abnormal lab value, does the system still respond safely with no internet at all) - re-run after every change, not written once and forgotten. Second, a small real end-to-end evaluation: 11 authored clinical test cases run through the whole pipeline, checking whether the final urgency level matches what it should be.

pytest -q, this repository, run fresh for this document; data/evaluation/test_cases.json

Q: What happens if there's no internet or no AI subscription at the venue - does the demo just fail?

A: No - this was built specifically so it doesn't. If the AI service is unreachable, unconfigured, or rate-limited, the system doesn't error out: the dangerous-symptom safety scan still runs with zero dependency on any AI service at all, and everything else falls back to a clearly-labeled, conservative "needs a human to check this" state rather than either crashing or silently guessing.

app/agents/triage.py: DeterministicFallbackReasoningBackend

Q: What's genuinely novel here, in terms a non-CS panel member would accept?

A: Most student health-chatbot projects are one AI model with a disclaimer. This has an independent, non-AI safety check that a real emergency word is never missed even if the AI is completely down; it asks different follow-up questions depending on what the complaint actually is (a skin rash and a cough get different, clinically real question sets, not the same script); and it's built against a real, currently-live national problem statement with real government-system integration attempted, not simulated.

Q: What are the current limitations, and what's the honest future scope?

A: Said plainly rather than left for a judge to find: the medical-guideline reference text is a starter set we wrote, not an official ICMR/WHO-certified document yet; the Ayurvedic assessment module needs a real AYUSH-trained reviewer before it can be trusted, and we say so rather than fake that expertise; OCR on a real messy handwritten prescription will read worse than a clean typed sample, and we've measured that gap rather than hidden it; and the national health-ID (ABDM) integration is a real first step (ID creation), not the full record-sharing pipeline, which is a multi-week government integration on its own. Next steps, in order: a domain expert review of the Ayurvedic module, a verified medical-guideline source, and testing OCR against real (not synthetic) prescription samples.

docs/sih/SIH26047_STRATEGY.md, Section D

1. Technical basics — what a more technical judge asks before opening any code

Q: What does this actually do, in one sentence?

A: Turns what a patient says, types, speaks, or photographs before their visit into a structured, physician-ready history - Chief Complaint through Prior Investigations - plus an independent safety-net triage level, and hands both to the doctor to review, never to decide.

Q: Who is this actually built for?

A: A patient in a 2-5 minute Indian primary-care consult (BMJ Open, 2017, 67-country consultation-length study - cited on our own landing page, not invented) and the physician who has to make a real decision inside that same window.

Q: What's the stack, and why so plain?

A: FastAPI + Pydantic backend, one hand-written JS file for the frontend (no React/Vue, no build step), plain sqlite3 for persistence, scikit-learn TF-IDF for guideline retrieval, Anthropic Claude when a key is configured, deterministic Python everywhere a model isn't. Every one of those is a named, deliberate choice, not a default - see section 2.

Q: Does it diagnose or prescribe?

A: No, structurally, not just by disclaimer: every schema field this pipeline outputs is either a triage level (self_care / clinic_visit / urgent / emergency) or a narrative history field. There is no diagnosis field, no medication-recommendation field, anywhere in app/schemas.py to fill in even by mistake.

2. Design parameters — every "why" a judge points at a specific choice

Q: Why TF-IDF for guideline matching, not embeddings or a vector DB?

A: In plain terms first: this is keyword-overlap matching (does the patient's wording share real medical terms with a known guideline), not a bigger AI model doing the matching - a deliberate choice, not a shortcut we ran out of time to fix. The corpus is a few dozen short, domain-specific chunks - at that scale, exact/near-exact medical-term overlap ("chest pain", "slurred speech") is already a strong signal, and a transformer embedding model adds real latency and a heavy dependency for a retrieval problem this small doesn't need. We say in the code exactly when this stops being the right call: hundreds of documents, or paraphrased non-overlapping wording.

app/agents/verify.py, GuidelineIndex docstring

Q: Why min_similarity = 0.2 for a guideline match?

A: Measured, not picked: a genuinely relevant chunk scores ~0.6-0.7 cosine similarity in our corpus; a chunk sharing only one incidental word ("mild") scores ~0.09-0.10. 0.2 sits cleanly between those two clusters, and that specific claim has its own named regression test, not just a comment.

app/agents/verify.py: top_matches(); test_top_matches_filters_out_weak_incidental_overlap

Q: Why does guideline verification only ever escalate, never de-escalate a level?

A: An imperfect TF-IDF match downgrading a real emergency is a far worse failure than staying cautious. Asymmetric on purpose - proven, not just claimed, in a test named exactly for that property.

app/agents/verify.py: verify_triage_decision(); test_verify_never_deescalates_even_with_a_mild_top_match

Q: Why best-match-only (k=1), not top-3?

A: We shipped top-3-most-severe-wins first, and it broke in live testing: a weak top-1 match on ordinary knee pain let a barely-above-threshold EMERGENCY chest-pain chunk ranked second silently override the correct verdict. Found by testing the live pipeline against ordinary phrasing, not a unit test in isolation - fixed to best-match-only the same day.

app/agents/verify.py, verify_triage_decision docstring, 12 Sep 2026 entry

Q: Why a keyword scan AND an LLM AND a guideline check - isn't that redundant?

A: Three independently-failing mechanisms on purpose: the keyword scan runs before any model call and can't have "a bad day"; the LLM covers paraphrase and nuance the fixed term list can't; the guideline check is a third, separately-computed opinion with a number the physician can actually see. All three would have to fail at once to miss a real emergency.

app/agents/intake.py: scan_red_flags(); app/agents/triage.py; app/agents/verify.py

Q: Why SOCRATES, and why four more question templates on top of it?

A: The PS text names SOCRATES specifically for pain. But SOCRATES doesn't fit a rash ("does it radiate?"), a cough, or a fever - asking every complaint the same eight pain questions is exactly the one-size-fits-all shortcut a clinically literate judge notices. So: real deterministic keyword classification into five categories (pain/general, respiratory, dermatological, GI, general-systemic), each with its own real clinical question set - not a pain template with nouns swapped.

app/agents/socrates_intake.py: classify_complaint_category(), generate_followup_questions()

Q: Why build deterministic fallbacks instead of just returning an error with no API key?

A: A rural kiosk with a dead connection, or a demo with no key configured, cannot 503 a sick patient. The fallback always proposes a fixed URGENT with confidence=0.0 - never a guessed self_care, never a guessed emergency - and every field it touches is honestly flagged requires_manual_triage=True downstream, never silently indistinguishable from a real judgment.

app/agents/triage.py: DeterministicFallbackReasoningBackend; app/schemas.py: ReferralResult.requires_manual_triage

Q: Why regex-based extraction for prescriptions/labs, not a trained NER model?

A: An in-house clinical NER model is out of reach in the time available, and a heuristic honestly labeled "candidates for a human to confirm" is safer than an ML-flavored black box that confidently gets a drug name wrong. We measured this ourselves: our own image-preprocessing step once turned "AMOXICILLIN" into "ANTONICILLIN" on a synthetic test image - kept in the docs, not hidden.

app/models/ocr.py: extract_medication_mentions(), _preprocess() docstring

Q: Why offline ASR/TTS at all, given how inaccurate they measurably are?

A: Zero-network, zero-cost availability at a PHC with no data connection beats hard-refusing voice entirely. We don't hide the ceiling: every offline-ASR transcript is flagged requires_manual_triage=True, and the module states its own measured accuracy floor from a controlled test, not a marketing claim.

app/adapters/offline_speech.py module docstring

Q: Why plain sqlite3, not an ORM or a hosted database?

A: One table, a fixed already-known shape, queried by primary key or a simple ORDER BY - exactly the case an ORM buys nothing for, while adding a dependency this scope doesn't need. Revisit the moment real multi-table relations or serious concurrent-write throughput actually show up - neither is true yet.

app/db.py module docstring

3. What's actually different from another team's prototype

Q: What's different from "a GPT wrapper that asks about symptoms"?

A: Six concrete, checkable things: (1) an escalate-only safety net structurally separate from the LLM's own answer; (2) a deterministic red-flag scan that runs before any model call and can't be prompted out of an emergency; (3) every triage decision carries a real, quantified guideline match (source + cosine-similarity score) a physician can audit, not a black-box verdict; (4) it keeps working with zero API key and zero internet - a pure LLM wrapper simply stops; (5) real government-scheme integration (ABDM ABHA creation, AYUSH Dashavidha Pariksha) that a generic chatbot has no reason to build; (6) a persisted case record a physician can actually reopen at consult time - a chat window is not a patient record.

Q: In the world of ChatGPT, why would anyone use this instead?

A: Open ChatGPT and describe your symptoms: there is no independent check on its own answer, nothing persists for the physician to reopen, no quantified evidence trail, no offline fallback if the connection drops, no ABDM/AYUSH integration, and its safety behaviour can be prompt-engineered around because it's one model with no external check. We built one narrow job - safe triage-support before a 2-minute consult - several structurally independent ways, so a bad model day, a missing key, or a clever prompt doesn't silently produce a wrong triage level.

Q: Isn't "AI is a scribe, never the decision-maker" just a slide, not real?

A: It's a field, not a slide: every summary defaults to `is_reviewed_by_physician=False` and there is a real accept/amend/reject endpoint behind a real login gate - a physician who never opens that screen leaves every case sitting as an unreviewed draft, visibly, not silently promoted to final.

app/schemas.py: ClinicalHistorySummary.is_reviewed_by_physician; POST /cases/{id}/review

Q: What real government systems does this actually touch, not just name-drop?

A: Bhashini (MeitY's own ASR/translation/TTS service) with the real two-step pipeline-config-then-inference request shape; ABDM's real V3 sandbox ABHA-enrollment flow with genuine RSA-OAEP encryption matching its published cipher spec (proven by a test that decrypts our own outgoing request with a real generated keypair). Both are honestly labeled: built and orchestration-tested, never confirmed against a live server, because neither service's live credentials exist in our build environment.

app/adapters/bhashini.py; app/adapters/abdm.py

4. Brutal — the questions a judge asks after reading the code, not the slide

Every answer here concedes the real limit first. That is the point: a rehearsed deflection is what actually loses trust in the room.

Q: Your "4/4 emergency recall, 4/4 accuracy" - is that a real statistic?

A: It's a real fraction, deliberately never shown as a percentage - we changed that ourselves after a live user pointed out a 4-case sample dressed up as "100%" was misleading. Our authored 11-case eval set has 5 true emergencies; without a live ANTHROPIC_API_KEY only the 4 red-flag-catchable ones are evaluable at all, and the UI says so rather than hiding the skip.

data/evaluation/test_cases.json; web/app.js: computeSafetyMetricsCounts()

Q: Is your guideline corpus real ICMR/WHO content?

A: No - stated in the module's own docstring: "a starter set written for this repo, not a verified extract from an official ICMR/WHO document." The retrieval mechanism is real and tested; sourcing a vetted corpus behind it is the next real step, not something we're presenting as already done.

app/agents/verify.py module docstring; data/guidelines/seed_guidelines.json

Q: TF-IDF isn't reasoning - is "Guideline-Verification Agent" overselling it?

A: It's lexical similarity, not clinical reasoning, and we'll say that before you ask. What it buys is a genuinely independent, auditable second check with a real number attached - worth more at this corpus size than a bigger model that just agrees with itself.

Q: You mention a Groq backend - is multi-vendor failover actually live?

A: No. It's built and tested behind the same interface as the Anthropic backend, but the live app's automatic fallback chain today is Anthropic to deterministic only - Groq is not wired into app/main.py. We're not claiming a failover path we haven't shipped.

app/agents/groq_backends.py (tested, not called from app/main.py)

Q: Has your ABDM integration ever actually talked to a real ABDM server?

A: No, and the code says so: no sandbox credentials exist in this environment, so the request/response shapes are transcribed from public documentation, not confirmed live. What IS verified without a live server: the encryption step ABDM's real system requires (scrambling the patient's ID and OTP with a public key before sending them, so only ABDM's own server can unscramble them) is genuinely correct, not just present - a test generates a real key pair, hands our code the public half exactly like the real ABDM sandbox would, and then uses the matching private half to unscramble our code's actual outgoing message and confirm it comes back byte-for-byte identical to the original.

app/adapters/abdm.py module docstring; tests/test_abdm.py

Q: What actually happens on a real handwritten prescription, not your clean demo image?

A: Measurably worse, and we tested it ourselves rather than assume: the identical extraction regex against a low-resolution rendering corrupted "13.0-17.0" into "130-170" - a wrong reference range, faithfully reported because the regex did exactly what it was asked. That's exactly why every extracted field is documented as a candidate for a human to confirm, never a source of truth on its own.

app/models/ocr.py: extract_lab_values() docstring, real measured Tesseract comparison

Q: Physician login is one shared passcode - what about per-doctor accounts or audit logs?

A: Real, named gap: one shared staff passcode, in-memory session tokens with no expiry, no per-physician identity. Correct scope for an MVP demo; the first thing that has to change before any real deployment, and we say that ourselves rather than wait for a judge to find it.

app/main.py: _PHYSICIAN_SESSIONS, require_physician_session()

Q: Does the referral engine pick the actual nearest hospital?

A: No - the first entry in a curated, static directory for that triage level, not real-time geolocation ranking. Stated in the referral agent's own docstring as an honest MVP scope, not implied as something smarter.

app/agents/referral.py: run_referral()

Q: Is voice input/output actually good enough for a real patient?

A: Input: real and working end-to-end, but PocketSphinx's accuracy on genuine speech is measurably modest - we flag every offline-ASR result requires_manual_triage=True rather than trust it silently. Output: intelligible but mechanical-sounding by construction - espeak-ng is a formant synthesizer, not neural TTS - and we checked for a better offline model before claiming otherwise: it exists (Piper), but its voice files live on huggingface.co, which is blocked in our build environment.

app/adapters/offline_speech.py module docstring, VOICE-QUALITY CAVEAT

Q: If I feed deliberately broken input - invisible Unicode, double spaces, weird whitespace - does it break?

A: We went looking for exactly that, repeatedly, across this build: six-plus real invisible-Unicode and whitespace bugs found and fixed across intake, triage parsing, and history drafting - each with a regression test proving it failed before the fix was written. Not a claim; it's in the commit history and the test suite.

app/schemas.py: _visible_length(); app/agents/intake.py: scan_red_flags(); app/agents/triage.py: _strip_invisible()

Q: Bottom line - what would you tell a judge who says "this is just another AI symptom checker"?

A: Open the code with them. A symptom checker has one model and a disclaimer. This has a deterministic safety net the model can't override, a quantified evidence trail, government-scheme integration, a persisted physician record, and it keeps working when the model doesn't - and every one of those claims is sitting in a file we can point to, not a slide.